Safety-based Termination Analysis With Transition Invariant Generation

Konstantin Britikov[1](✉), Martin Blicha, Grigory Fedyukovich[2], and Natasha Sharygina[1]

[1] University of Lugano, Lugano, Switzerland
[2] Florida State University, Tallahassee, FL, United States

Abstract. Termination analysis is difficult.

1 Preliminaries

This section introduces the definitions and formalisms used for the construction of the Termination Analysis techniques. Logical formulas in our approach are defined over the theory of Linear Integer Arithmetic (LIA). Conventional primed notation is used to represent "next-state" variables. In cases where there are multiple states, we use superscripts to distinguish them, e.g., $x^{(0)}, x^{(1)}, \ldots$. We use $s(x), s_1(x), s_2(x) \ldots$ to denote the specific states of the system, which is a conjunction in which each variable in x is assigned a specific value.

Transition systems and Safety Problem. A *transition system* is defined as $T = \langle Init, Tr, x \rangle$, where $Init$ is an initial state of the system represented by a first-order logic formula, Tr is a transition formula, which represents the transition in the system, and x is a set of system's variables. A *safety problem* is defined as $P = \langle Init, Tr, Bad, x \rangle$ where Bad is a formula that represents a state violating the safety property. Safety verification amounts to determining whether there exists a transition trace through the system that reaches a BadBad Bad state. Safety problem P is unsafe if there exists a satisfying assignment for the following formula:

$$Init(x^{(0)}) \wedge Tr(x^{(0)}, x^{(1)}) \wedge \ldots \wedge Tr(x^{(n-1)}, x^{(n)}) \wedge Bad(x^{(n)})$$

If P is SAFE, then there exists an inductive invariant $Inv(x)$ such that $Init(x) \implies Inv(x)$, $Inv(x) \wedge Tr(x, x') \implies Inv(x')$, and $Inv(x) \implies \neg Bad(x)$. We use $safetyCheck(P)$ to denote a safety verification procedure which returns a tuple $\langle res, \phi \rangle$ where $res = SAFE$ if TS is safe and $\phi = Inv(x)$ (inductive state invariant), otherwise if TS is unsafe $res = UNSAFE$ and ϕ is a trace that leads to the bad states:

$$\phi = trace(x^{(0)}, \ldots, x^{(n)}) = Init(x^{(0)}) \wedge Tr(x^{(0)}, x^{(1)}) \wedge \ldots \wedge Tr(x^{(n-1)}, x^{(n)}) \wedge Bad(x^{(n)}).$$

Nontermination. A transition system is called **nonterminating** if there exists a set of states, called a *recurrent set*, within which there always exists SAT transition to another state in recurrent set.

Definition 1 (Recurrent Set). *For a Transition System $\langle Init, Tr, x\rangle$, a recurrent set is a formula $N(x)$ such that:*

- $\forall x.N(x) \implies \exists x'.Tr(x, x') \wedge N(x')$
- $\exists x.N(x) \wedge Init(x)$

Intuitively, every state in the recurrent set has a successor that also lies in the recurrent set, and the set is reachable from some initial state.

Termination. A Transition System is called **Terminating** if for every initial state there does not exist an infinite satisfiable transition trace. Termination can be proven by synthesizing a *ranking function* [3] , which maps states to a well-founded domain such that its value strictly decreases along every transition. Additionally, a *transition invariant* [4] can be used to prove termination.

Definition 2 (Transition invariant). *For a Transition System $\langle Init, Tr, x\rangle$, $TrInv(x, x')$ is a transition invariant if $Tr^*(x, x') \subset TrInv(x, x')$ over the set of reachable states. In a First Order Logic: $\forall x, x'.R(x) \wedge Tr^*(x, x') \wedge R(x') \implies TrInv(x, x')$, where $Tr^*(x, x')$ is the reflexive transitive closure of $Tr(x, x')$ and $R(x)$ is a formula which describes all of the states, reachable within transition system.*

Transition invariant proves the termination of transition system, if it can be expressed as a finite disjunction of well-founded relations.

Definition 3 (Disjunctively well-founded transition invariant). *A transition invariant $TrInv(x, x') = R_1(x, x') \vee R_2(x, x') \vee \cdots \vee R_n(x, x')$ is called disjunctively well-founded if every disjunct $R_i(x, x')$ is well-founded.*

Craig Interpolation. This is a technique for computing an overapproximation of the formula [2]. Particularly, given two formulas A and B such that $A \wedge B$ is unsatisfiable, a Craig interpolant for A and B is a formula I such that $A \implies I$, $I \wedge B$ is unsatisfiable, and I only contains the symbols that are common to both A and B. We use $Itp(A, B)$ to denote the Craig interpolant for A and B.

Proving Nontermination via Safety. Several techniques reduce termination and nontermination analysis to safety verification. One notable approach is the reduction of nontermination to safety, which constructs recurrent sets via inductive invariants [1]. It was introduced in the context of proving nontermination of programs with nested loops, and adapts naturally to the setting of transition systems.

Definition 4 (Sink states). *For a Transition System $\langle Init, Tr, x\rangle$, a state $Sink(x)$ is a sink state, if $Sink(x) \wedge Tr(x, x')$ is unsatisfiable. We use $Sink(x)$ to denote the formula describing all sink states in the transition system:*

$$\forall x.Sink(x) \implies \neg\exists x'.Tr(x, x')$$

Algorithm 1: Nontermination analysis using Safety.

Input : Safety problem $P = \langle Init, Tr, Sink, x \rangle$
Output: NONTERM — UNKNOWN

1 **while** $\top$ **do**
2 $\quad \langle res, \phi \rangle \leftarrow safetyCheck(P)$
3 $\quad$ **switch** $safetyCheck(P)$ **do**
4 $\quad\quad$ **case** $\langle UNSAFE, trace(x^{(0)}, \ldots, x^{(n)}) \rangle$ **do**
5 $\quad\quad\quad M \models trace(x^{(0)}, \ldots, x^{(n)})$
6 $\quad\quad\quad i \leftarrow n - 1$
7 $\quad\quad\quad$ **while** $i > 0$ **do**
8 $\quad\quad\quad\quad s_i(x) \leftarrow$ *variable assigment extracted from* M for $x^{(i)}$
9 $\quad\quad\quad\quad s_{i+1}(x) \leftarrow$ *variable assigment extracted from* M for $x^{(i+1)}$
10 $\quad\quad\quad\quad$ **if** $SAT\ ?\ s_i(x) \wedge Tr(x, x') \wedge \neg s_{i+1}(x')$ **then**
11 $\quad\quad\quad\quad\quad Tr(x, x') = Tr(x, x') \wedge \neg s_{i+1}(x')$
12 $\quad\quad\quad\quad$ **else**
13 $\quad\quad\quad\quad\quad i \leftarrow i - 1$
14 $\quad\quad\quad$ **if** *i=0* **then** $Init(x) \leftarrow Init(x) \wedge \neg s_0(x)$
15 $\quad\quad$ **case** $\langle SAFE, Inv(x) \rangle$ **do**
16 $\quad\quad\quad$ **if** $\forall x. Inv(x) \implies \exists x'. Tr(x, x') \wedge Inv(x')$ **then**
17 $\quad\quad\quad\quad$ **return** NONTERM
18 $\quad\quad\quad$ **return** UNKNOWN

Sink states can be computed by applying quantifier elimination to $\neg \exists x'. Tr(x, x')$. It is possible to construct the following safety problem: $P = \langle Init, Tr, Sink, x \rangle$. The algorithm for proving nontermination of transition system is as follows:

The algorithm begins with a safety check of P, where the bad states are set as the Sink states. At line 5, the algorithm handles the case when P is unsafe (i.e., a trace reaching a sink state exists). The safety checker returns a concrete trace $trace(x^{(0)}, \ldots, x^{(n)})$, and the algorithm extracts the satisfying model, that allows to calculate the formulas $s_0(x), s_1(x), \ldots, s_n(x)$ - representing the reached system state after each step of the trace. Using these formulas, the algorithm iteratively checks if it is possible to take a transition from $s_i(x)$ to a state, different from $s_{i+1}(x)$. If it is possible, then the algorithm blocks the state $s_{i+1}(x)$ from the transition (at line 11), since this state inevitably leads to a sink state. Otherwise, if such $s_i(x)$ does not exist, then the state $s_0(x)$ is deterministically leading to the sink states, indicating that all executions from s_0 terminate; accordingly, s_0 is excluded from the initial states.

In the case when P is safe, the algorithm extracts the inductive invariant $Inv(x)$ at line 15. The algorithm checks whether $Inv(x)$ is a recurrent set at line 17, meaning that it is possible to take a transition from every state within the invariant to another state. If $Inv(x)$ is a recurrent set, then the transition system is nonterminating. Otherwise, algorithm returns UNKNOWN, as the analysis is inconclusive.

2 Termination Analysis with Interpolation-based Transition Invariant Generation

In this section, we present our approach for safety-based termination analysis, which extends the Safety-based Nontermination Analysis (SNA) to prove termination of transition systems and to improve the nontermination analysis. We begin with with the description of the updates to the original transition-system-based approach.

2.1 Termination of Transition Systems

Algorithm 1 can be extended to prove termination of transition systems via a single additional SMT check. Specifically, when the algorithm reaches the SAFE case at line 15, the following check is performed:

$$Init(x) \wedge Tr(x, x')$$

This check is intended to determine whether it is possible to take a transition after all traces to sink states have been blocked. Throughout the execution of the algorithm, the transition relation is progressively refined to exclude states that deterministically lead to sink states. If this formula is unsatisfiable, then the transition system is terminating (since Init states lead to sink states).

2.2 Safety Problem Refinement

The algorithm can further refine the safety problem P in the case where Inv does not constitute a recurrent set. Particularly, it is possible to extract states within the invariant, from which it is impossible to take a transition (which makes Inv not a recurrent set).

$$\forall x.Sink'(x) \implies Inv(x) \wedge \neg\exists x'.Tr(x, x')$$

In this formula, $Sink'(x)$ is a new formula which describes sink states within the inductive invariant. It can be computed by applying quantifier elimination to $\neg\exists x'.Tr(x, x')$. Afterwards, the produced formula $Sink'(x)$ replaces $Sink(x)$ in the safety problem P, enabling further analysis.

After those two updates the algorithm would look as follows:

This algorithm differs from the original one in two lines. Particularly, at line 16 the algorithm checks if one can identify states within the invariant from which no transition is possible — precisely those states that prevent Inv from being a recurrent set. It allows to check the termination in some cases. And at line 15 the algorithm refines the sink states, by extracting states from the invariant, from which it is impossible to take a transition. Which allows to proceed with termination analysis and allows to solve more complex problems.

Algorithm 2: Nontermination analysis using Safety.

Input : Safety problem $P = \langle Init, Tr, Sink, x \rangle$
Output: NONTERM — UNKNOWN

```
while ⊤ do
    ⟨res, φ⟩ ← safetyCheck(P)
    switch safetyCheck(P) do
        case ⟨UNSAFE, trace(x^(0), ..., x^(n))⟩ do
            M ⊨ trace(x^(0), ..., x^(n))
            i ← n − 1
            while i > 0 do
                s_i(x) ← variable assigment extracted from M for x^(i)
                s_{i+1}(x) ← variable assigment extracted from M for x^(i+1)
                if SAT ? s_i(x) ∧ Tr(x, x') ∧ ¬s_{i+1}(x') then
                    Tr(x, x') = Tr(x, x') ∧ ¬s_{i+1}(x')
                else
                    i ← i − 1
            if i=0 then Init(x) ← Init(x) ∧ ¬s_0(x)
        case ⟨SAFE, Inv(x)⟩ do
            if UNSAT ? Init(x) ∧ Tr(x, x') then return TERM
            if ∀x.Inv(x) ⟹ ∃x'.Tr(x, x') ∧ Inv(x') then return NONTERM
            Sink(x) ← ∀x.Sink'(x) ⟹ Inv(x) ∧ ¬∃x'.Tr(x, x')
```

References

1. Chen, H.Y., Cook, B., Fuhs, C., Nimkar, K., O'Hearn, P.W.: Proving nontermination via safety. In: Ábrahám, E., Havelund, K. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings. pp. 156–171. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_11, https://doi.org/10.1007/978-3-642-54862-8_11
2. Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. The Journal of Symbolic Logic **22**(3), 269–285 (1957)
3. Podelski, A., Rybalchenko, A.: A complete method for the synthesis of linear ranking functions. In: Steffen, B., Levi, G. (eds.) Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings. pp. 239–251. Lecture Notes in Computer Science, Springer (2004). https://doi.org/10.1007/978-3-540-24622-0_20, https://doi.org/10.1007/978-3-540-24622-0_20
4. Podelski, A., Rybalchenko, A.: Transition invariants. In: 19th IEEE Symposium on Logic in Computer Science (LICS 2004), 14-17 July 2004, Turku, Finland, Proceedings. pp. 32–41. IEEE Computer Society (2004). https://doi.org/10.1109/LICS.2004.1319598, https://doi.org/10.1109/LICS.2004.1319598